FMU-ZWIERS-ECUADOR / BEAMNODE_PROTOTYPE2

BEAM Overhaul Plan

How the system works today, what reading the code turned up, the new branch-and-review workflow, and a first milestone that fixes the riskiest problems without changing how the sensors behave.

Plan only: no code changed yet | Updated 2026-10-15 | Full plan: beam_overhaul.md

WHAT WE NEED TO DO

Next steps, in order

- **Admin access is needed.** Branch protection and the default-branch change can only be made by a repository admin (`pzwiersFMU` or `RAIZ44`). Everyone else on the team, including Jackson, has write access.

PHASE 0 · SET UP THE WORKFLOW

1 **Agree on the plan and the open decisions** Whole team

Read this page, then settle the open questions at the bottom: approvals per pull request, admin bypass, and the format for node settings.

2 **Create the `dev` branch** Jackson

Branch `dev` from the current `main` . Write access is enough for this.

3 **Open the first pull request into `dev`** Jackson

Add `docs/WORKFLOW.md` , a pull request template and a Contributing section in the README. Change `repull.sh` and the clone instructions to `git clone -b main` . Another developer reviews it.

4 **Release it to `main`** Jackson Reviewer

Open a `dev` → `main` pull request, get one approval, and merge it. From then on, nodes always clone `main` , whatever the default branch is.

5 **Turn on protection and make `dev` the default** Admin

Only after step 4. If `dev` becomes the default before `repull.sh` pins `main` , nodes would start running `dev` . The commands are in the admin section.

6 **Switch to the new habits** Whole team

No more pushing to `main` and no commits on a Pi straight to `main` or `dev` . Every change goes on an initials branch and is reviewed by someone else.

BEFORE THE MILESTONE · HOTFIXES AND CLEANUP

7 **Fix the scheduler timeout crash** `Live bug`

Catch the timeout and give each sensor its own limit, with audio getting `duration_sec + 30`. Today scheduled audio never saves, and other sensors can stop for up to an hour.

8 **Fix `calibrate_ph.py`** `Bug`

It writes pH calibration results into the `atlas_do` section. It should write to `atlas_ph`.

9 **Clean up the repository** `Any developer`

One pull request into `dev` that deletes the stale copies in `supervisor/` (keep `SupervisorGateway_Setup.sh`), the unused `models/`, the stray `models~…` file, every `.DS_Store` and `.vscode/`, and ignores them in `.gitignore`. **Keep the `logs/` folder** as `logs/.gitkeep` and ignore only its contents: `beamnode.service` writes there, and after `repull.sh` re-clones, a missing folder stops the service from starting. Details in repo structure.

FIRST MILESTONE

10 **Build the milestone in 10 small pull requests** `Team`

Each step from the milestone list gets its own branch and its own reviewed pull request into `dev`.

11 **Test on one node, then release** `Team`

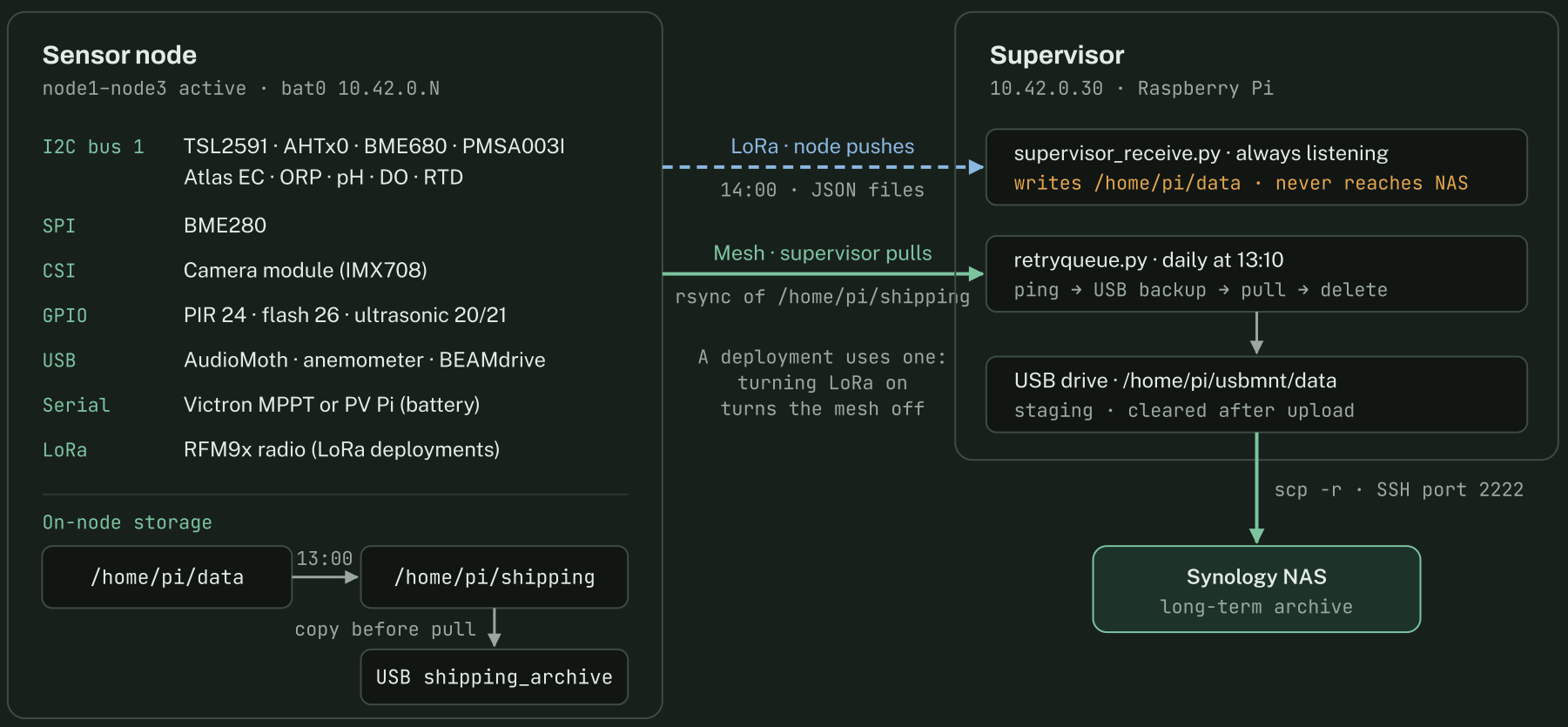
Migrate one node, reboot it, force low power, trigger the PIR during shipping, and check the results. Only then release to `main` and roll out to the rest of the fleet.

HOW IT WORKS TODAY

Our setup

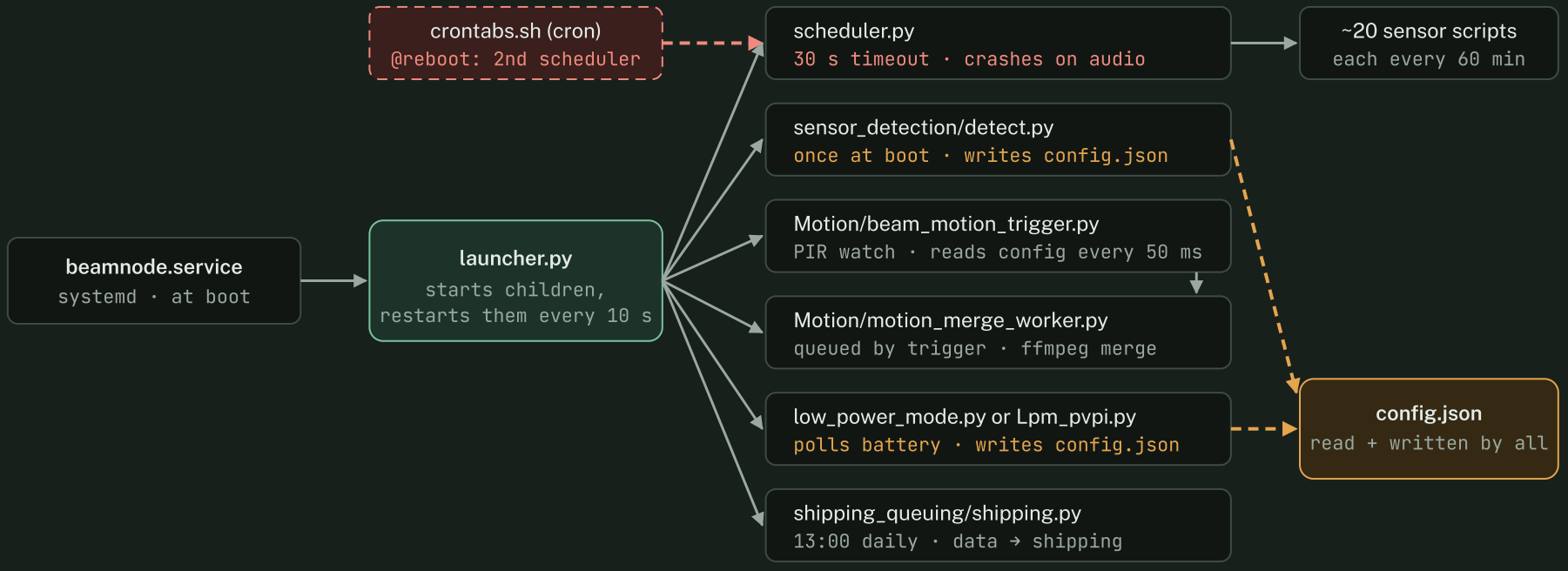
Every diagram in this section and the next is drawn from the code on `main`, not from older documentation.

Hardware and network



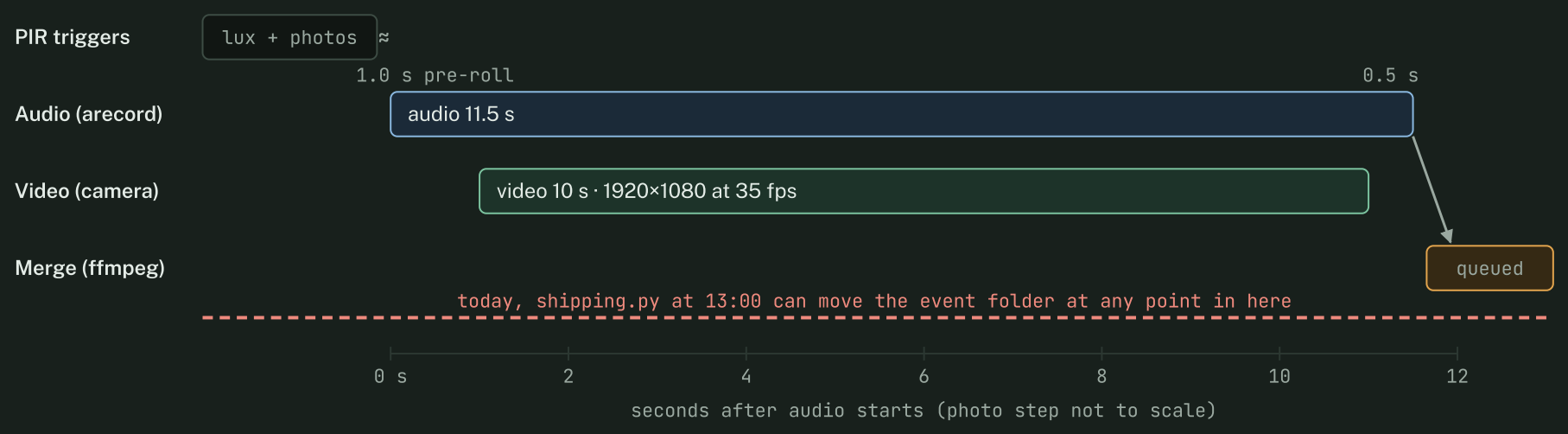
Each deployment moves data home one of two ways. With the **mesh**, the supervisor logs into every node and pulls its shipping folder. With **LoRa**, each node pushes its JSON files over radio. The NAS is the only long-term archive. The supervisor's USB drive is just a staging area, and the node's own USB keeps a copy of everything it shipped.

What runs on a node



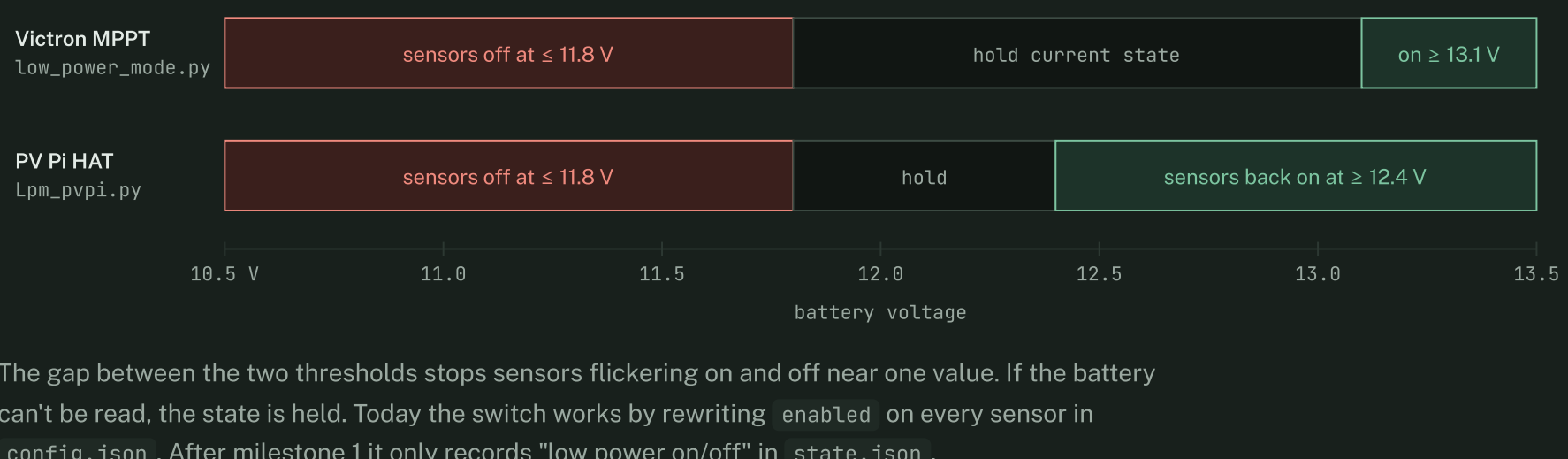
The launcher owns most processes, but **cron starts a second scheduler** on reboot. Settings, detected hardware and live power state all live in one `config.json`, and several of these processes rewrite it while others read it.

One motion event, second by second



Timings come from the current `config.json`: `duration_sec` 10, `audio_preroll_sec` 1.0, `audio_postroll_sec` 0.5. The merge runs later in the background, so an event folder stays incomplete for a while after the camera stops.

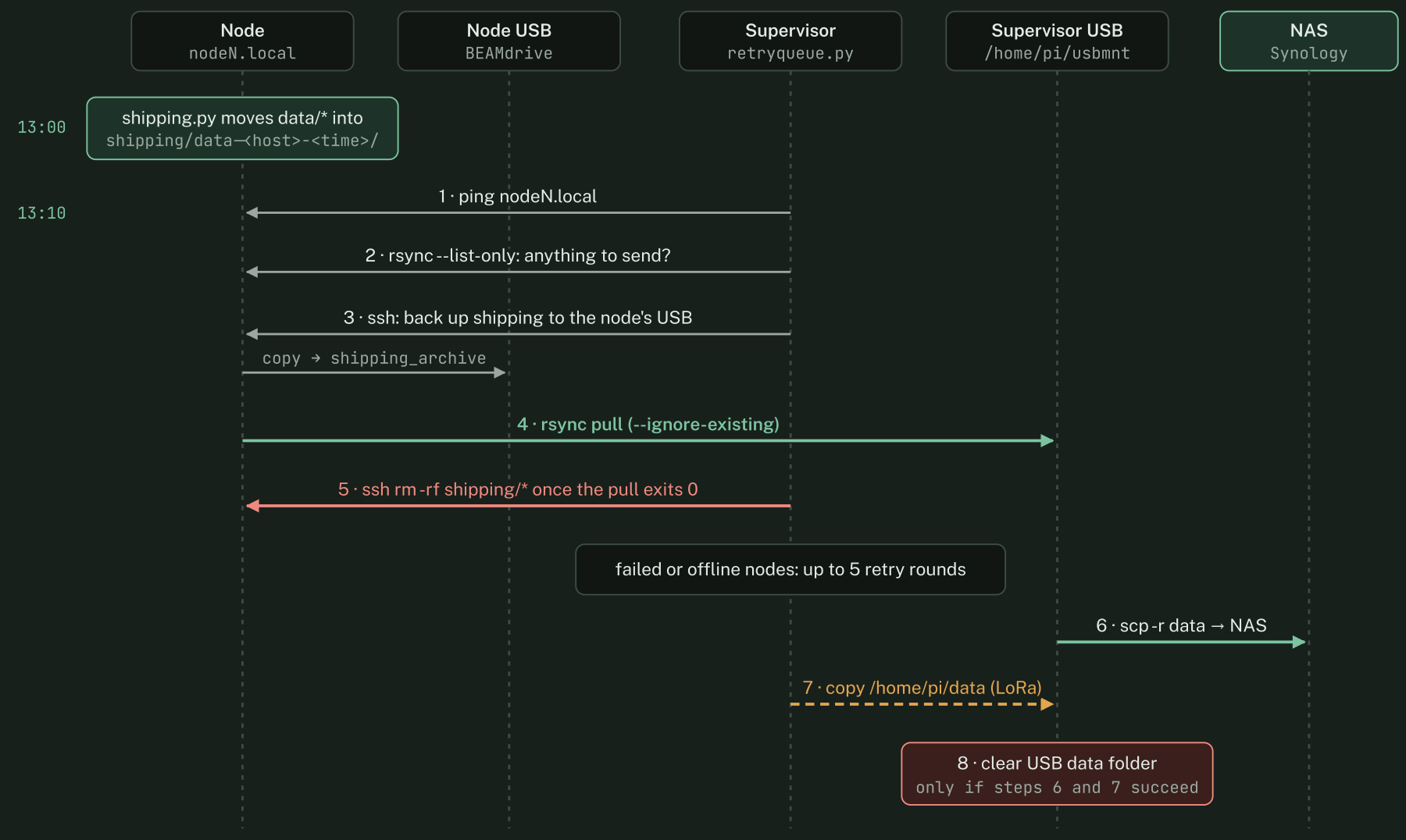
When low-power mode switches



The gap between the two thresholds stops sensors flickering on and off near one value. If the battery can't be read, the state is held. Today the switch works by rewriting `enabled` on every sensor in `config.json`. After milestone 1 it only records "low power on/off" in `state.json`.

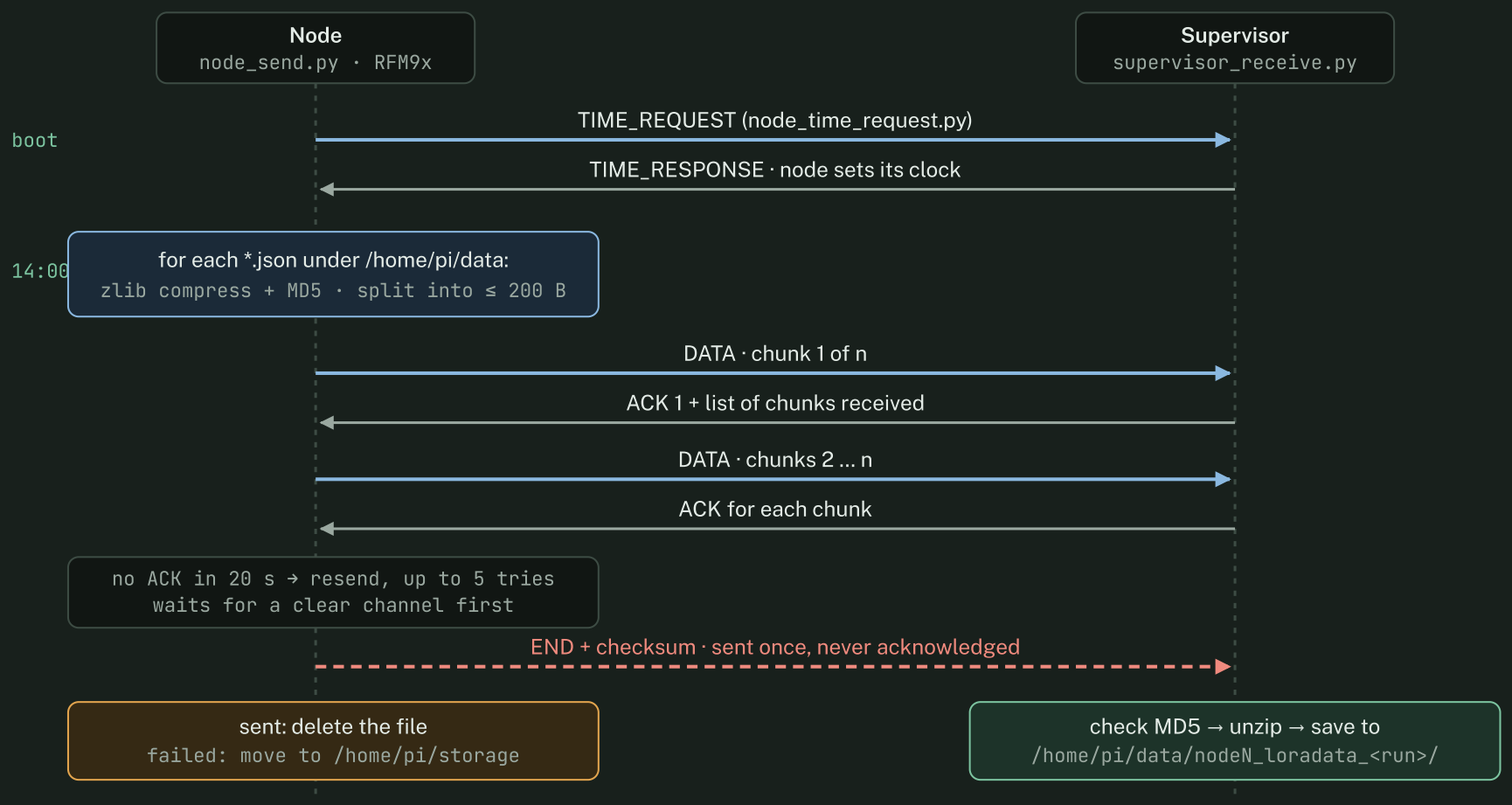
How data gets home

Mesh deployments: the daily transfer



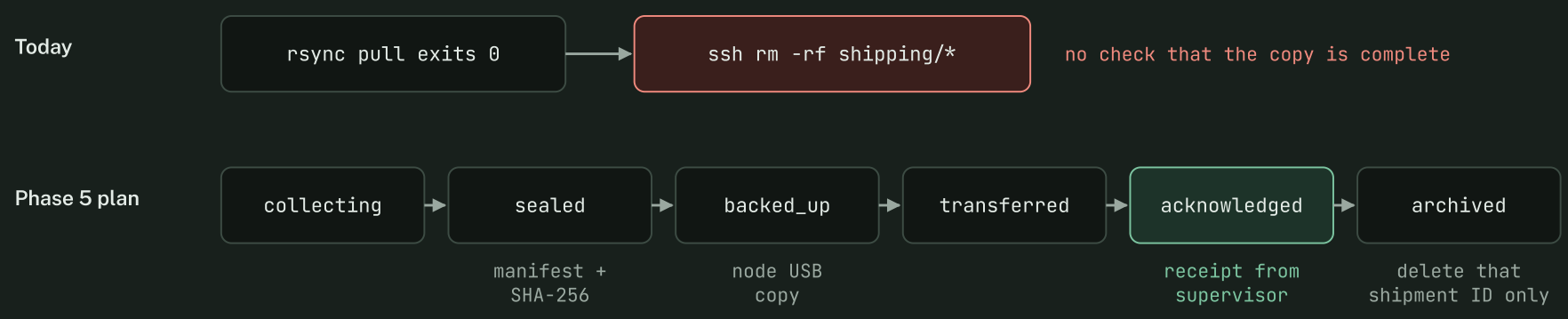
Two weak points. **Step 5** deletes the node's whole shipping folder on the strength of rsync's exit code alone; nothing checks that every file arrived. **Step 7** copies the supervisor's LoRa folder onto the USB *after* the NAS upload, and step 8 then clears it, so LoRa data never reaches the NAS.

LoRa deployments: sending one file



Every chunk is acknowledged, but the final **END** packet is not. If END is lost, or the supervisor's checksum fails, the supervisor drops the file while the node has already deleted its copy. LoRa also sends *every* JSON file under `/home/pi/data`, including motion merge jobs and metadata, and deletes them afterwards.

When is it safe to delete? Today vs. Phase 5



The difference is the **receipt**. The supervisor confirms the shipment ID, file count, byte count and manifest checksum before the node deletes anything. The same receipt works for the mesh, LoRa, USB and the NAS.

WHY

What reading the code turned up

The original overhaul doc described the problems in general terms. Checking it against the code showed several are already losing data in the field.

Data loss

Scheduled audio never saves

Audio records for 300 s, but the scheduler kills anything still running after 30 s. The timeout error is not caught either, so the scheduler crashes and then waits until the next hour to restart.

Data loss

Motion events shipped mid-capture

`shipping.py` moves everything at 13:00, including events still recording. Their audio/video merge jobs are left pointing at files that have moved, so those events are never merged.

Data loss

LoRa data never reaches the NAS

The supervisor copies its LoRa folder onto the USB only after the NAS upload, then clears the USB. The only copy is left in `/home/pi/data` on the supervisor.

Data loss

LoRa can delete unsaved files

The final END packet is never acknowledged. If it's lost, or the checksum fails, the node still deletes its copy. LoRa also sends and deletes motion merge jobs and metadata.

Battery

Reboot during low power turns sensors back on

Sensor detection re-enables everything it finds at boot. The power monitor still thinks low power is already handled, so it never turns those sensors off again.

Reliability

Motion capture keeps stopping

The PV Pi monitor rewrites `config.json` in place every 10 s. The motion trigger reads it about every 50 ms, and when it catches a half-written file it shuts down.

Settings

Node settings get wiped

`repull.sh` deletes and re-clones the repo, and each node's settings live inside it. Detection also overwrites `enabled` on every boot, so a connected sensor can't be kept off.

Reliability

Shipping deletes on exit code alone

The supervisor runs `rm -rf shipping/*` as soon as rsync exits 0. Nothing checks that every file arrived before the node's copy is gone.

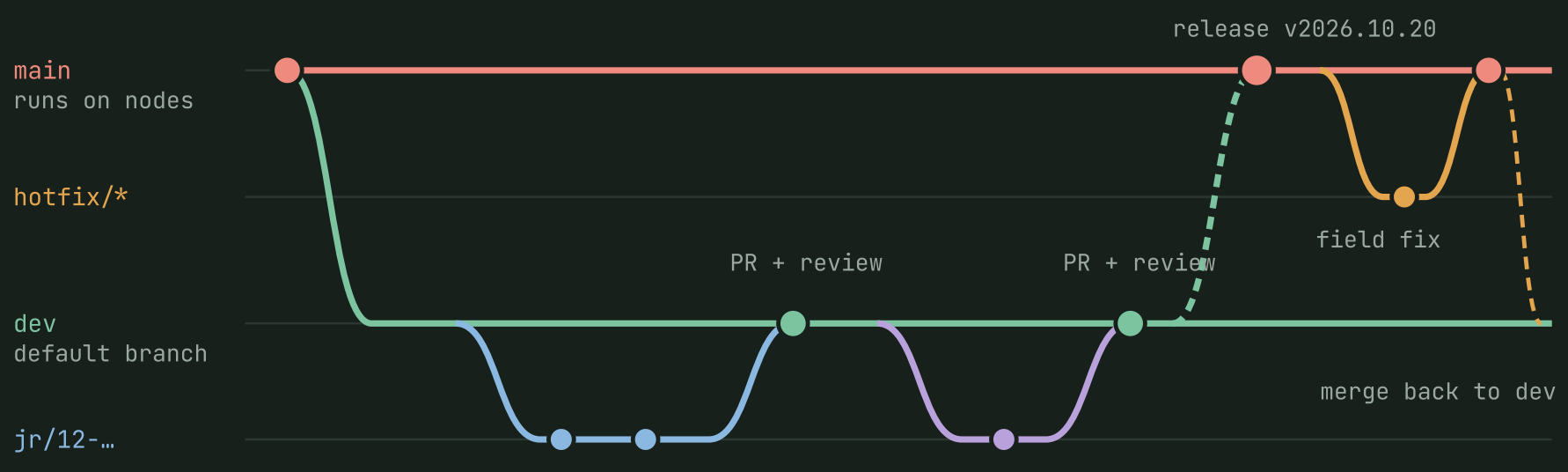
Also found: two schedulers can run at once (cron and the launcher both start one), `America/New_York` is hard-coded on nodes deployed in Ecuador, two motion events in the same second share a folder, and 356 lines across 95 files hard-code `/home/pi` or the repository path.

PHASE 0

The new workflow

Nobody pushes to `main` or `dev` directly. Work happens on your own branch, someone else reviews it, and `main` only changes through a release.

Branches



Work branches merge into `dev` after review. `dev` is released into `main`, which is what the nodes pull. Urgent fixes branch from `main` and are merged back into `dev`.

BRANCH	WHAT IT IS	HOW IT CHANGES
`main`	What runs on deployed nodes and supervisors. Always deployable.	Reviewed release PR from `dev`, or a reviewed hotfix
`dev`	Integration branch and the repo's default branch	Reviewed PRs only
`jr/12-scheduler-timeout`	One branch per piece of work: your initials, the issue number, a short name. Use `jr/scheduler-timeout` when there is no issue.	The owner pushes freely
`hotfix/usb-perms`	An urgent fix for deployed nodes, branched from `main`	Reviewed PR into `main`, then merged back into `dev`

Everyday flow

1. Pick or create an issue.
2. Branch from the latest `dev`: `git switch dev && git pull && git switch -c jr/12-scheduler-timeout`
3. Commit in small steps and push your branch.
4. Open a pull request into `dev`. Say what changed, how you tested it, and which node you tested on if hardware is involved.
5. **Another developer reviews it.** GitHub won't let you approve your own pull request. If you push again after an approval, it needs a fresh one.
6. Resolve every comment, then squash-merge. The branch is deleted automatically.

On the Raspberry Pis

- Never commit on a Pi directly to `main` or `dev`. If you have to change something on a Pi, make a branch there (`git switch -c nc/fix-usb-perms`), push it, and open a pull request.
- Deployed nodes stay on `main`. To try a branch on a node, run `git fetch && git switch <branch>`, then return with `git switch main && git pull`.

- Releases are tagged with their date (for example `v2026.10.20`), so a node can be rolled back to a known version.

FOR PZWIERSFMU OR RAIZ44

Repository settings

Run these with the `gh` CLI **after** the `repull.sh` fix has been released to `main` (step 4 above).

1. Protect `main` and `dev`

```
for branch in main dev; do
gh api -X PUT "repos/fmu-zwiers-ecuador/BEAMNode_Prototype2/branches/$branch/protection" --input - <<'EOF'
{
  "required_status_checks": null,
  "enforce_admins": false,
  "required_pull_request_reviews": {
    "required_approving_review_count": 1,
    "dismiss_stale_reviews": true,
    "require_code_owner_reviews": false,
    "require_last_push_approval": true
  },
  "restrictions": null,
  "required_conversation_resolution": true,
  "allow_force_pushes": false,
  "allow_deletions": false
}
EOF
done
```

2. Make `dev` the default and tidy up merging

```
gh api -X PATCH repos/fmu-zwiers-ecuador/BEAMNode_Prototype2 \
  -f default_branch=dev \
  -F delete_branch_on_merge=true \
  -F allow_squash_merge=true \
  -F allow_merge_commit=true \
  -F allow_rebase_merge=false
```

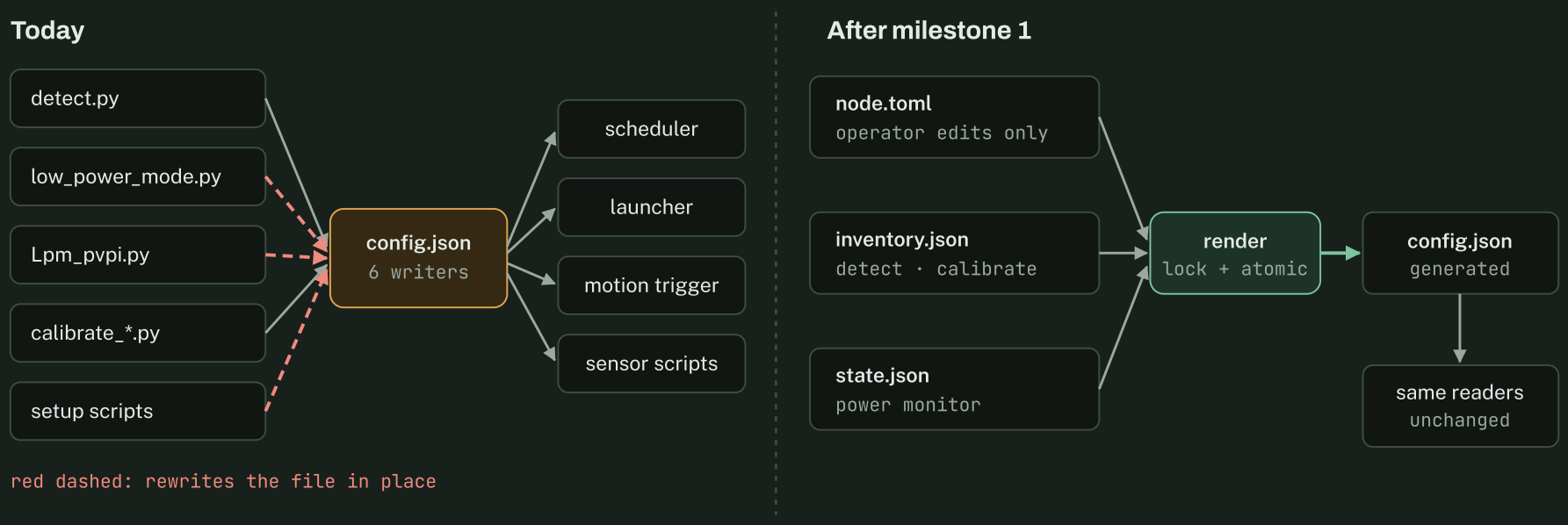
- `require_last_push_approval` means the last push must be approved by someone other than the person who pushed it. This is what makes review by another developer required.
- `enforce_admins: false` lets admins bypass the rules in a real field emergency. Bypasses should be rare and announced to the team.
- When CI is added in Phase 1, list its checks under `required_status_checks` on both branches.

FIRST MILESTONE

Split settings from live state, and stop shipping half-finished events

Today one file, `config.json`, holds the operator's choices, detected hardware, and live power state, and six different scripts write to it. The milestone splits it into three files with clear owners and generates `config.json` from them. The scripts that only read it keep working without changes.

Who writes the configuration



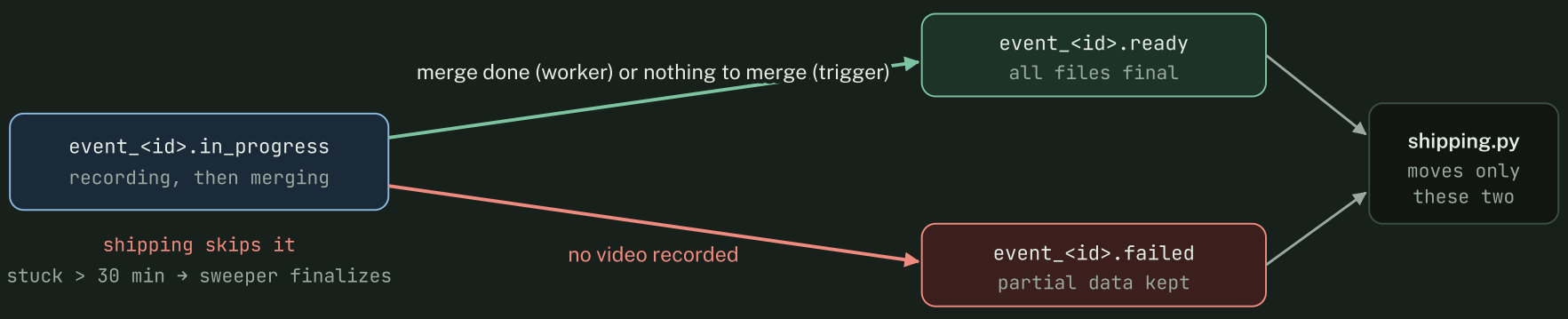
`node.toml` lives in `/etc/beamnode/`; `inventory.json` and `state.json` live in `/var/lib/beamnode/`. Node settings move out of the repository, so `git pull` and `repull.sh` can no longer wipe them.

Whether a sensor runs is calculated, never written back

```
run sensor = configured AND detected AND NOT low_power
```

A dead battery or an unplugged cable no longer changes anyone's settings. When power comes back, exactly the right sensors restart. A sensor the operator turned off stays off. Migration sets every sensor to `enabled = true`, which means "run when detected", so each node behaves exactly as it does today.

Motion event lifecycle



Each event also gets an `event.json` manifest listing its files and their status. Event IDs include milliseconds and the node ID, so two events in the same second no longer share a folder.

The 10 steps, each its own pull request into `dev`

#	STEP	WHAT IT COVERS
1	Scaffolding	`pyproject.toml`, `src/beamnode/`, test folder and fixtures
2	Paths and locking	One `RuntimePaths` object with environment-variable overrides; safe file writes that also work on Windows
3	Config schema	Typed validation that lists every problem at once, plus `node.example.toml`
4	Inventory, state, render	The three files, the run-sensor rule, and the generated `config.json`
5	Migration tool	Converts a node's current `config.json`; `--check` shows any difference before anything is written

#	STEP	WHAT IT COVERS
6	Switch the writers	detect, both power monitors, calibration and the setup scripts write to the new files; `config.json` is no longer tracked in git
7	Power tests	Thresholds, the hold band, restarts, and the reboot-during-low-power regression
8	Motion manifests	Event lifecycle and `event.json`; the merge worker marks events ready
9	Shipping gate	`shipping.py` moves only finished events and reports errors; LoRa skips motion events
10	Docs	Configuration guide and data formats

Not touched in this milestone: sensor scripts, camera and audio code, mesh and LoRa transfer, and systemd. The LoRa and NAS transfer bugs above are Phase 5 work. All tests run without hardware, on Windows or Linux.

REPOSITORY STRUCTURE

The GitHub repo today, and where it is going

The left tree is what `main` tracks today. The right tree is the target layout from `beam_overhaul.md`, reached over Phases 1 to 6. Arrows show where today's code ends up. Milestone 1 only adds files; nothing moves until Phase 3.

164 tracked files, 35 of them Markdown spread over 20 folders

3 copies of the BATMAN mesh setup

2 different `retryqueue.py` files; only one is installed

13.6 MB YOLO model that no script loads

95 files that hard-code `/home/pi` or the repo path

Today vs. target

Today

main · 2026-10-15

```
BEAMNode_Prototype2/
├── README.md  SSH_Setup_Guide.md  linux_cheatsheet.md
├── repull.sh  monitor_internet.sh  restart_internet.sh
├── wlan1.sh  wpa_supplicant-wlan1.conf
├── beamnode.service  batman_mesh_info.txt
├── models/                  13.6 MB, never loaded
├── models~20b2f3…           stray empty file
├── logs/                    runtime logs in git
├── .vscode/  .DS_Store ×9
├── docs/                    8 per-script write-ups
├── installation_bash/       setup scripts + legacy/
├── scripts/
│   ├── node/
│   │   ├── launcher.py  scheduler.py  crontabs.sh
│   │   ├── config.json      settings+hardware+state
│   │   ├── Motion/          trigger · merge · capture
│   │   ├── camera/          older motion code, unused
│   │   ├── sensor_detection/  detect.py
│   │   ├── atlas_sci/       16 scripts: log · calibrate
│   │   ├── ahtx0/ air_quality/ anemometer/ audio/
│   │   ├── bme280/ bme680/ tsl2591/ ultrasonic/
│   │   ├── node_gateway/
│   │   └── shipping_queuing/
│   │       ├── shipping.py      node · 13:00
│   │       ├── retryqueue.py    supervisor · the live one
│   │       └── retrylogic.sh  ping_nodes_10min.py
│   ├── power/               Victron · PV Pi monitors
│   ├── lora/                node + supervisor + units
│   └── Batman_mesh/         setup copy 1 of 3
└── supervisor/
    ├── data_collection/     older retryqueue.py
    ├── mesh_setup/          setup copy 2 of 3
    └── set_retryservice.sh  copy of installation_bash
```

Target

after Phase 6

```
BEAMNode_Prototype2/
├── README.md                + Contributing
├── pyproject.toml           deps, extras per role
├── .gitignore               logs, data, caches
├── .github/
│   ├── pull_request_template.md
│   └── workflows/ci.yml     tests, no hardware
├── src/beamnode/            the installable package
│   ├── paths.py  locking.py
│   ├── config/              schema · inventory · state
│   ├── cli/                 ← launcher.py, detect.py
│   ├── scheduling/          ← scheduler.py
│   ├── hardware/            camera · gpio · i2c · lora
│   ├── sensors/             ← the per-sensor folders
│   ├── motion/              ← Motion/
│   ├── power/               ← scripts/power/
│   ├── storage/             ← shipping.py
│   ├── transport/           ← scripts/lora/ + rsync
│   └── supervisor/          ← retryqueue.py
├── config/
│   ├── node.example.toml    replaces config.json
│   └── supervisor.example.toml
├── deploy/
│   ├── systemd/             ← every .service, .timer
│   ├── ansible/             ← installation_bash/, mesh
│   └── network/             ← root wlan + repull
├── tests/                   unit · integration · hardware
├── docs/                    ← all 35 .md, by topic
└── legacy/                  retired code, never run

Off the repo, on each Pi:
/etc/beamnode/node.toml        operator settings
/var/lib/beamnode/             inventory · state
```

remove from the repo · duplicate or in the wrong place · new · where today's code moves

Two things the tree hides. The supervisor's live `retryqueue.py` sits under `scripts/node/`; that is the copy `set_retryservice.sh` installs. The one in `supervisor/data_collection/` is an older version (last changed 2026-05-29). The third copy of the BATMAN setup is written inline inside `installation_bash/node_setup.sh`, which uses neither folder. `deploy/network/` and the calibration commands in `cli/` are this page's suggestion; the rest follows `beam_overhaul.md`.

How the tree changes, phase by phase



The order protects the fleet. The cleanup and Milestone 1 change nothing a node runs by path. CI and the docs reorganization, the rest of Phase 1, follow the milestone. Phase 3 moves the code but leaves a small wrapper at every old path, so `launcher.py`, the scheduler's `script_name` lookups, cron and the systemd units keep working. Only Phase 6, when deployment owns the units, removes those wrappers.

Where each part of today's tree goes

TODAY	GOES TO	WHEN
`scripts/node/config.json`	`/etc/beamnode/node.toml` and `/var/lib/beamnode/` on each Pi; `config/node.example.toml` in the repo. A generated `config.json` stays for the old readers.	Milestone 1
`models/`, `models~20b2f3…`, `.DS_Store` ×9, `.vscode/`	Deleted and added to `.gitignore`	Phase 1, early
`logs/`	Log files deleted, folder kept as `logs/.gitkeep`, contents ignored. `beamnode.service` appends to `logs/launcher.log` and can't create the folder itself. Moves to `/home/pi/logs` in Phase 6.	Phase 1, early
`supervisor/data_collection/`, `supervisor/mesh_setup/`, `supervisor/set_retryservice.sh`	Deleted. They are older or identical copies; nothing installs them.	Phase 1, early
`scripts/node/camera/`	`legacy/`, after confirming no node still runs it	Phase 1
`docs/` and the Markdown in 19 other folders	`docs/` by topic: architecture, configuration, data formats, deployment, troubleshooting, `adr/`	Phase 1 on
`launcher.py`, `scheduler.py`, `sensor_detection/detect.py`	`src/beamnode/cli/` and `scheduling/`; probe code to `hardware/`	Phase 3
`ahtx0/` `air_quality/` `anemometer/` `audio/` `bme280/` `bme680/` `tsl2591/` `ultrasonic/`, `atlas_sci/log_*.py`	`src/beamnode/sensors/`, one adapter per sensor behind the common `Sensor` interface	Phase 3
`atlas_sci/calibrate_*.py`, `switch_*_to_i2c.py`	Subcommands of `cli/`, e.g. `beamnode calibrate ph`	Phase 3
`scripts/power/`	`src/beamnode/power/`: Victron and PV Pi readers plus one shared threshold rule	Phase 3
`PIRTEST.py`, `test_*.py`, `Testing.py`, `testlux.py`, `Camtest`	`tests/hardware/`, kept out of the normal test run	Phase 3
`scripts/node/Motion/`	`src/beamnode/motion/`: controller, capture, exposure, merge worker	Phase 4
`shipping_queuing/shipping.py`	`src/beamnode/storage/`, with shipment manifests	Phase 4–5

TODAY	GOES TO	WHEN
`shipping_queuing/retryqueue.py`, `retrylogic.sh`, `ping_nodes_10min.py`, `scripts/lora/*.py`	`src/beamnode/supervisor/` and `transport/` (mesh and LoRa behind one interface)	Phase 5
`beamnode.service`, `beam-motion-merge.service`, `lora-*.service` / `.timer`, `batman.service`, `crontabs.sh`	`deploy/systemd/`; cron entries become systemd timers	Phase 6
`installation_bash/`, `scripts/Batman_mesh/`, `supervisor/SupervisorGateway_Setup.sh`	`deploy/ansible/` roles: common, node, supervisor, mesh, LoRa, storage	Phase 6
`repull.sh`, `wlan1.sh`, `wpa_supplicant-wlan1.conf`, `monitor_internet.sh`, `restart_internet.sh`, `node_gateway/`	`deploy/network/` until Ansible takes them over	Phase 6

Rules for moving files

History

Use `git mv`, one area per PR

A move in its own pull request, with no edits mixed in, keeps `git log --follow` working and keeps the review readable. "Move power monitors" is one PR; "move motion" is another.

Nodes

Leave a wrapper at the old path

The scheduler looks scripts up by folder and `script_name`, and 95 files hard-code `/home/pi` or the repo path. Until Phase 6, every moved script leaves a two-line stub behind that imports from `beamnode`.

Naming

Rename to snake_case only when moving

`Motion/`, `Batman_mesh/`, `Lpm_pvpi.py` and `ultrasonic/Detect.py` get fixed as they move. On Windows a case-only rename takes two steps: `git mv Motion motion_tmp`, then `git mv motion_tmp motion`.

Size

Deleting the model won't shrink clones

The repo history is 57 MB and the model stays in it after deletion. Removing it from `main` still saves 13.6 MB of checkout on every Pi. Rewriting history isn't worth the risk to everyone's clones.

BIG PICTURE

Where this fits

Phase 0 · now

Workflow

Branches, review, protection

Milestone 1 · next · covers Phase 2

Reliability

Config split, paths, power, motion events

Phase 1

Baseline

Cleanup first, then CI and docs

Phase 3

Package

Sensor, storage and transport interfaces

Phase 4–5

Data and transfer

JSON Lines or SQLite, shipment receipts

Phase 6

Deployment

systemd ownership, automated setup

NEEDS A DECISION

Open questions for the team

Approvals per pull request

Recommended: 1

Two would leave pull requests waiting during field season.

Admin bypass

Recommended: allowed, for emergencies only

Lets a fix reach a failing node when no reviewer is available.

Settings file format

Recommended: TOML

Operators can leave comments in it, and Python 3.11 on the Pis reads it without extra packages.

Ship failed motion events?

Recommended: yes, after 30 minutes

Partial recordings still reach the supervisor rather than sitting on the node.

Delete the copies in `supervisor/` ?

Recommended: yes, in the cleanup PR

`data_collection/` , `mesh_setup/` and `set_retryservice.sh` are older or identical copies. The supervisor runs `scripts/node/shipping_queuing/retry_queue.py` . Keep `SupervisorGateway_Setup.sh` , the only original file there.

Keep the YOLO model?

Recommended: remove from `main`

No script loads it. If someone wants it for a future experiment, keep it on the NAS or attach it to a GitHub release.

Retire `scripts/node/camera/` ?

Recommended: move to `legacy/`

Motion capture now runs from `Motion/` . Nothing in the launcher, scheduler, cron or setup scripts calls these files.

Code owners

Optional

Automatically request a reviewer who knows the hardware for changes in `scripts/power/` or `scripts/lora/` .

This page summarizes `beam_overhaul.md` , which has the full findings, module list and test plan. Nothing has been changed in the repository or on GitHub yet.